FinCore — What You Need to Know (Business, Idea & Skills)

Oct 22, 2026 · @Mohamed

The project idea

FinCore is the engine that sits behind products like Paymob or Stripe: it accepts payment requests from merchants, talks to an external payment provider, records every movement of money in a ledger that cannot be rewritten, and proves its books agree with the provider's and the bank's.

Every design decision serves three promises:

1. **Never lose money.** Every accepted payment is durably recorded, even if a server crashes mid-request.
2. **Never create money.** Every ledger entry balances: debits equal credits.
3. **Never charge twice.** Retries, duplicate webhooks and race conditions cannot move money more than once.

What FinCore is not: it does not store card numbers (only provider tokens), does not connect to a real bank (a simulator plays the provider), and has no frontend. The goal is correctness, not features.

Five invariants you will test in every phase:

1. Sum of debits equals sum of credits, per journal entry and globally.
2. Ledger rows are never updated or deleted; corrections are new rows.
3. A payment moves only along legal state transitions.
4. The same idempotency key and same payload give the same result, exactly once.
5. Every external fact (webhook, provider report) is stored raw before it is interpreted.

The business explained

A payment platform sits between merchants, their customers and the banks, and earns a small fee on every payment it moves.

Party	Role	Example
Merchant	The business that wants to get paid; FinCore's customer	An online shop
Payer	The person paying the merchant	A shopper
PSP / acquirer	Connects to Visa, Mastercard or Meeza; FinCore integrates one (simulated)	An acquiring bank
Issuing bank	The payer's bank, which holds and then releases the funds	The card issuer
FinCore	The middle layer: payment API, ledger, reconciliation	Paymob, Stripe

How one payment flows

1. The merchant asks FinCore to create a payment (nothing moves yet).
2. The payer pays with a token; the provider authorizes, which means the issuing bank holds the funds.
3. The merchant captures; this is the moment revenue becomes real.
4. The provider settles to the bank, usually 1 to 3 days later, minus its fee.
5. FinCore pays the merchant its share.
6. A refund is a new, opposite transaction. Nothing is ever deleted.

How FinCore earns. In the blueprint, FinCore charges the merchant 2.5% and the provider charges FinCore 1.5%. On a 250.00 EGP payment, the merchant fee is 6.25 EGP, the provider fee is 3.75 EGP, and FinCore keeps 2.50 EGP.

The mindset shift. The money customers pay does not belong to FinCore; it belongs to the merchant, so it is a liability. Only the fee is revenue. Engineers who miss this treat customer money as company income.

Words you will meet daily

Term	Plain meaning
Authorization	The bank agrees to hold the funds; no money has moved
Capture	The merchant claims the held funds

Term	Plain meaning
Settlement	The provider actually transfers the money, net of fees
Refund	Returning captured money; a new opposite transaction
Ledger	The permanent book of every financial fact
Journal entry	One business event as balanced debit and credit lines
Idempotency key	A client token that makes a retry safe
Webhook	An asynchronous callback telling you something happened
Outbox	A table that guarantees an event is published if and only if the DB transaction committed
Reconciliation	Comparing your records with the provider's and the bank's, and explaining every difference

Accounting you must know

Every financial event is a journal entry made of lines; each line debits or credits an account, and total debits must equal total credits. Debit and credit are directions, not "minus and plus"; what they do depends on the account type.

Account type	Increases with	FinCore example
Asset (what we hold or are owed)	Debit	Provider Clearing (1000), Bank Settlement (1100)
Liability (what we owe)	Credit	Merchant Payable (2000)
Revenue (what we earn)	Credit	Fee Revenue (4000)
Expense (what we spend)	Debit	Provider Fees (5000)

Worked example: a 250.00 EGP payment (25,000 piasters), merchant fee 2.5%, provider fee 1.5%

Step	Account	Debit	Credit
1. Capture	1000 Provider Clearing	25,000	
	2000 Merchant Payable		24,375
	4000 Fee Revenue		625
2. Settlement	1100 Bank Settlement	24,625	
	5000 Provider Fees	375	
	1000 Provider Clearing		25,000
3. Payout to merchant	2000 Merchant Payable	24,375	
	1100 Bank Settlement		24,375
4. Refund of 100.00 EGP	2000 Merchant Payable	10,000	
	1000 Provider Clearing		10,000

Steps 1 to 3 net to a margin of 250 piasters: 625 fee revenue minus 375 provider fees, which is also what remains in the bank. Step 4 is a separate event: the refund moves only the gross amount because the fee is non-refundable by default.

Rules to memorise:

- Money is stored as whole piasters (long), never float or double.
- Line amounts are always positive; the side (debit or credit) carries the direction.
- Never update a ledger row. A mistake is fixed by a reversing entry plus a correct entry.
- Balance = sum of lines. A stored balance is only a projection that can be rebuilt.

The payment lifecycle and the unknown-outcome problem

A payment moves through defined states, and many moves are illegal: you cannot capture a failed payment or refund an uncaptured one. The state machine makes illegal moves impossible.

From	Allowed next states
Created	Processing, Cancelled, Expired
Processing	Authorized, Failed, PendingReview
Authorized	Captured, Cancelled (void), Expired
Captured	PartiallyRefunded, Refunded, Settled
PartiallyRefunded	PartiallyRefunded, Refunded, Settled
Failed, Cancelled, Expired, Refunded, Settled	None (terminal)

The hardest problem in payments. You send an authorization request and your HTTP call times out. Did the money get held or not? You do not know. Marking the payment Failed risks charging the customer for nothing; retrying blindly risks charging twice. The correct behaviour:

1. Never conclude failure from a timeout.
2. Move the payment to PendingReview (outcome unknown).
3. Resolve it by asking the provider (inquiry by your own reference) or by receiving its webhook.
4. Always send your own reference and an idempotency key to the provider so a retry cannot create a second authorization.

Never hold a database transaction open during a network call. Transaction 1 records the attempt and sets Processing; then the network call happens; Transaction 2 records the outcome. If the process dies in between, a sweeper finds the stuck payment and runs an inquiry.

Webhooks are hostile. Assume they can be forged (verify an HMAC signature), duplicated (deduplicate by event id), out of order (let the state machine decide) and slow (store the raw event, answer 200, process later).

Technical skills you should have

SQL Server and concurrency are the two skills that decide whether this project succeeds. Everything else you can learn along the way.

Skill	Level to start	Needed from phase
C#: OOP, async/await, dependency injection, records and value objects	Solid	1
ASP.NET Core: middleware, filters, authentication handlers, hosted services, ProblemDetails	Solid	1
SQL Server and T-SQL: constraints, indexes, transactions, isolation levels, triggers, UPDLOCK and READPAST, stored procedures	Strong; the most important skill	2
EF Core: fluent configuration, value converters, migrations, concurrency tokens, interceptors, global query filters	Working	1
HTTP and REST: status codes, headers, cursor pagination, API versioning	Solid	1
Testing: xUnit, FluentAssertions, Testcontainers, FsCheck property tests	Working	1
Concurrency: race conditions, optimistic and pessimistic locking, parallel tests with Task.WhenAll	Working	2
Cryptography basics: SHA-256, HMAC, constant-time comparison, why keys are hashed	Basic	1
Resilience: Polly, HttpClientFactory, BackgroundService, retries, circuit breakers	Working	5
Git and GitHub: branches, pull requests, Actions	Basic	1
Docker and Docker Compose (you need SQL Server in a container from day one)	Basic	1
Observability: Serilog, OpenTelemetry, Prometheus, Grafana	Basic	9

Skill	Level to start	Needed from phase
Security: OWASP API Top 10, JWT, RBAC	Basic	9
Performance: set-based SQL, bulk loading, k6 load testing	Basic	7

If you are weak in T-SQL transactions and isolation levels, spend your first week there before starting Phase 2.

Engineering concepts to understand before coding

You do not need to master these first, but you should be able to explain each in one sentence before the phase that uses it.

Concept	What it means	Phase
Money as integers	Store minor units (piasters) plus a currency code; avoids rounding drift	1
Aggregate and invariants	The domain object refuses to exist in an invalid state	1
Idempotency	Same key and same payload gives the same result, exactly once	2
Insert-first locking	Insert the key under a unique index so the race loser is detected by the database	2
State machine	Only listed transitions are legal; everything else is rejected	2
Optimistic concurrency	A rowversion check makes the second concurrent writer fail instead of overwriting	2
Double-entry	Every event balances; debits equal credits	3
Append-only and reversals	Never update; correct with a mirrored entry	4
Hash chain	Each entry stores a hash of the previous one, so tampering is detectable	4

Concept	What it means	Phase
Projection	A fast stored balance that can always be rebuilt from the ledger	4
Anti-corruption layer	Provider formats and codes are translated at the boundary	5
Circuit breaker, backoff, jitter	Fail fast when a dependency is down; retry with growing, randomised delays	5
HMAC signature	Proves a webhook came from someone who knows the shared secret	6
At-least-once delivery	Duplicates will happen, so handlers must be idempotent	6
Transactional outbox	Write the event in the same transaction as the change, publish later	6
Three-way matching	Compare your records, the provider's report and the bank statement	7
Leasing and dead letters	A dead worker's job is released automatically; poison messages are parked	8
BOLA	Merchant A must never read merchant B's data; return 404	9
Maker-checker	One person requests, a different person approves	9
Expand and contract	Backward-compatible schema changes so rolling deploys are safe	10

Study order and readiness check

Learn just before each phase rather than everything up front. Suggested order:

1. **Before Phase 1:** C# value objects and records, EF Core basics, Docker with SQL Server, how API keys and hashing work.
2. **Before Phase 2:** SQL Server transactions and isolation levels (lost update, write skew), unique-index race handling, Stripe's documentation on idempotent requests.
3. **Before Phase 3:** double-entry accounting basics; the Modern Treasury "Ledgers" article series.

4. **Before Phase 4:** SQL Server permissions and triggers, atomic updates and UPDLOCK; Designing Data-Intensive Applications chapter 7.
5. **Before Phase 5:** Polly, HttpClientFactory, and the stability patterns in Release It!.
6. **Before Phase 6:** HMAC signing, the Stripe webhook documentation, the outbox pattern.
7. **Before Phase 7:** reconciliation concepts, set-based SQL, SqlBulkCopy.
8. **Before Phases 8 to 10:** hosted services, sp_getapplock, OpenTelemetry, Prometheus, GitHub Actions.

Readiness check. You are ready to start when you can explain these out loud:

- [ ] Why 0.1 + 0.2 breaks money, and how minor units fix it
- [ ] What happens if two identical requests arrive at the same millisecond
- [ ] Why a customer's payment is a liability and not revenue
- [ ] Why you must not mark a payment Failed on a timeout
- [ ] What a debit and a credit do to an asset and to a liability
- [ ] Why a webhook can arrive twice, late or out of order

What you can skip or defer: Kafka, dbt, fraud rules, Kubernetes, Terraform and cloud certificates are not part of FinCore. Add them later only if you target data-heavy roles.

A realistic warning. The full blueprint is large (about 25 weeks of part-time work at this ticket breakdown). A finished Phase 1 to 8 beats an unfinished Phase 1 to 10, so tag a release at the end of every phase and keep the README updated.